# PS Laser: A Multi-Stack Web Platform for Instant Cost Estimation, Enquiry Handling and Order Tracking in a Laser-Cutting Job Shop

Rahul Diwate
*Dept. of Computer Engineering*
*Vishwakarma Institute of Technology*
*Pune-411037, Maharashtra, India*
rahul.diwate2@vit.edu

Rivan Shetty
*Dept. of Computer Engineering*
*Vishwakarma Institute of Technology*
*Pune-411037, Maharashtra, India*
rivan.shetty24@vit.edu

Gaurav Shende
*Dept. of Computer Engineering*
*Vishwakarma Institute of Technology*
*Pune-411037, Maharashtra, India*
gaurav.shende24@vit.edu

Vishal Sharma
*Dept. of Computer Engineering*
*Vishwakarma Institute of Technology*
*Pune-411037, Maharashtra, India*
vishal.sharma24@vit.edu

Gandharv Shinde
*Dept. of Computer Engineering*
*Vishwakarma Institute of Technology*
*Pune-411037, Maharashtra, India*
gandharv.shinde24@vit.edu

***Abstract*—In the sheet-metal job shop studied here, quotation requests arrive by telephone and WhatsApp, which delays pricing, scatters drawing files and gives customers no view of job progress. This paper presents PS Laser, a web platform built for a laser-cutting job shop at Velu, Bhor (Pune). It provides a public website, an instant indicative cost estimator, an enquiry and drawing-upload service, order tracking and a staff back office. The platform is implemented as seven cooperating services (PHP, Laravel, React with Redux, Servlet/JSP, Spring Boot, Next.js with TypeScript, and Node.js with Express) that share one MySQL schema and one written contract covering the price formula, JWT authentication and API shapes. The same estimate formula, with one rounding rule, is implemented in four languages and returned the same total (INR 2,933.83) for a reference part on all four stacks. We describe the architecture, the cost model, the security measures (JWT, bcrypt, CSRF tokens, prepared statements, upload validation, rate limiting) and a push channel for live work-order status. Verification consists of cross-stack equivalence checks, endpoint and validation tests, and automated suites (33 Node.js tests and 8 Laravel tests passing on 18 October 2026). The rate card is sample data and the system is neither deployed nor connected to shop-floor machines; these limits and future work are discussed.**

***Index Terms*—web technologies, cost estimation, laser cutting, sheet-metal fabrication, REST API, JWT, Spring Boot, Next.js, React, Laravel, MySQL, order tracking**

## I. INTRODUCTION

PS Laser is a sheet-metal and CNC laser-cutting job shop at Velu, Tal. Bhor, Pune. Its equipment comprises two fiber lasers (3 kW and 1.5 kW, 3 m × 1.5 m beds), a 175 t CNC press brake, two Amada turret punches (32 t and 16 t), a 1.5 kW handheld laser welder and a PLC-controlled powder-coating plant. The shop's published customer channels are the telephone and WhatsApp: a customer describes a part, sends a drawing, and waits for a manual quotation.

This channel has three practical weaknesses. First, a customer cannot get even an approximate price without waiting for a person. Second, drawings and requirements arrive as chat attachments that are hard to search, track or audit. Third, once a job is accepted the customer has no way to see whether it is being cut, bent, coated or ready for dispatch. These weaknesses are specific to the shop we studied; we do not claim they hold for every fabricator.

The platform described here addresses them with a mobile-friendly website, an indicative estimator, an enquiry form with drawing upload, a phone-number-verified order-tracking page and staff screens for managing enquiries, the rate card and work orders. The work was also carried out as the course project of CS3111 *Web Technologies*, whose six units cover PHP/Laravel, React, Servlets/JSP, Spring Boot, advanced React and Node.js. We therefore implemented one real feature per technology instead of a single monolith, and spent most of the design effort on keeping those independent services consistent with each other.

The contributions of this paper are: (i) a contract-first architecture in which seven services share one database schema, one JWT secret and one price formula; (ii) a rate-card cost model that gives identical results in PHP, Java, Spring Boot and TypeScript; (iii) a security and resilience design that includes a graceful offline fallback for the public site; and (iv) an honest account of what was verified, what was not, and which values are sample data.

## II. PROBLEM STATEMENT AND OBJECTIVES

**Problem statement.** Design and build a web platform that lets a laser-cutting job shop (a) present its real machines and services, (b) give customers an instant, clearly labelled indicative price, (c) capture enquiries and drawings reliably, (d) let customers follow a job to dispatch, and (e) let staff maintain prices and job status without editing the database by hand.

**Objectives.**

1. Provide a public website with the shop's machines, services and contact channels, with every "Get Quote" action opening a pre-filled WhatsApp message.
2. Compute an indicative estimate from material, thickness, size, cut length, pierces, bends, optional powder coating and quantity, using a rate card that the owner can change.

3. Store enquiries and drawing files (DXF, DWG, PDF, PNG, JPG, up to 10 MB) with server-side validation.
4. Expose order tracking that requires the order number and the customer's phone number, and a staff dashboard that updates live.
5. Keep all services consistent through one database, one authentication token format and one documented API contract.
6. Make the platform demonstrate every unit of the CS3111 syllabus with working code.

## III. BACKGROUND AND RELATED WORK

The platform relies on well-established web-engineering ideas. The REST architectural style [1] guides the Spring Boot and Node.js APIs, and the model-view-controller separation [2] organises the Laravel, Servlet/JSP and Spring code. Stateless authentication uses JSON Web Tokens [3] signed with HMAC-SHA-256 (the HS256 algorithm defined in [4]); passwords are stored as adaptive bcrypt hashes [5]. Live status updates use the WebSocket protocol [6] with a STOMP broker in Spring. The web-application risks that shaped the validation and access-control design are those catalogued by OWASP [7], and the quality attributes used in the requirements document follow ISO/IEC 25010 [8].

On the manufacturing side, laser-processing texts such as [9] describe the process parameters (laser power, material, thickness and cutting speed) that determine cut quality and throughput. Our estimator deliberately does not model that physics. It is a rate-card model, in which each cost component is a quantity multiplied by an owner-set rate, because a shop needs a quick, explainable and editable price indication rather than a process simulation.

*Scope of the literature review. We did not identify directly comparable, verifiable publications on small-shop quotation portals and therefore make no novelty claim about the estimator itself. The contribution lies in the integration and consistency of the multi-stack design. A systematic comparison with commercial online-quotation tools is left as future work.*

## IV. PROPOSED METHODOLOGY AND SYSTEM DESIGN

### *A. Contract-first, shared-schema approach*

Before any service was written, a contract document (`services/CONTRACT.md`) fixed the database name and tables, the JWT claims and secret, the estimate formula and its JSON shape, the REST endpoints of the Spring and Node services, and the allowed CORS origins. Each service was then built against that document. The deliberate trade-off is that PHP, Laravel and the Java servlet application read and write MySQL directly instead of calling the Spring API. This reduces the number of network hops and lets each syllabus unit stand alone, but it duplicates some business rules, which is why the price formula is specified once and tested in every stack.

### *B. Architecture*

Fig. 1 shows the architecture as built. The client tier is a browser used by customers and staff. The presentation tier has five front ends: the Next.js website (port 3001), the React portal (5173), the PHP estimator (8000), the Laravel admin panel (8001) and the Servlet/JSP desk on Tomcat (8080). The application tier consists of the Spring Boot REST API (8081) and the Node.js/Express API (4000). The data tier is MySQL/MariaDB (3306) with MongoDB (27017) used only for an audit activity log. The physical shop is shown for context only: no machine is connected to the software, and staff update job status by hand.

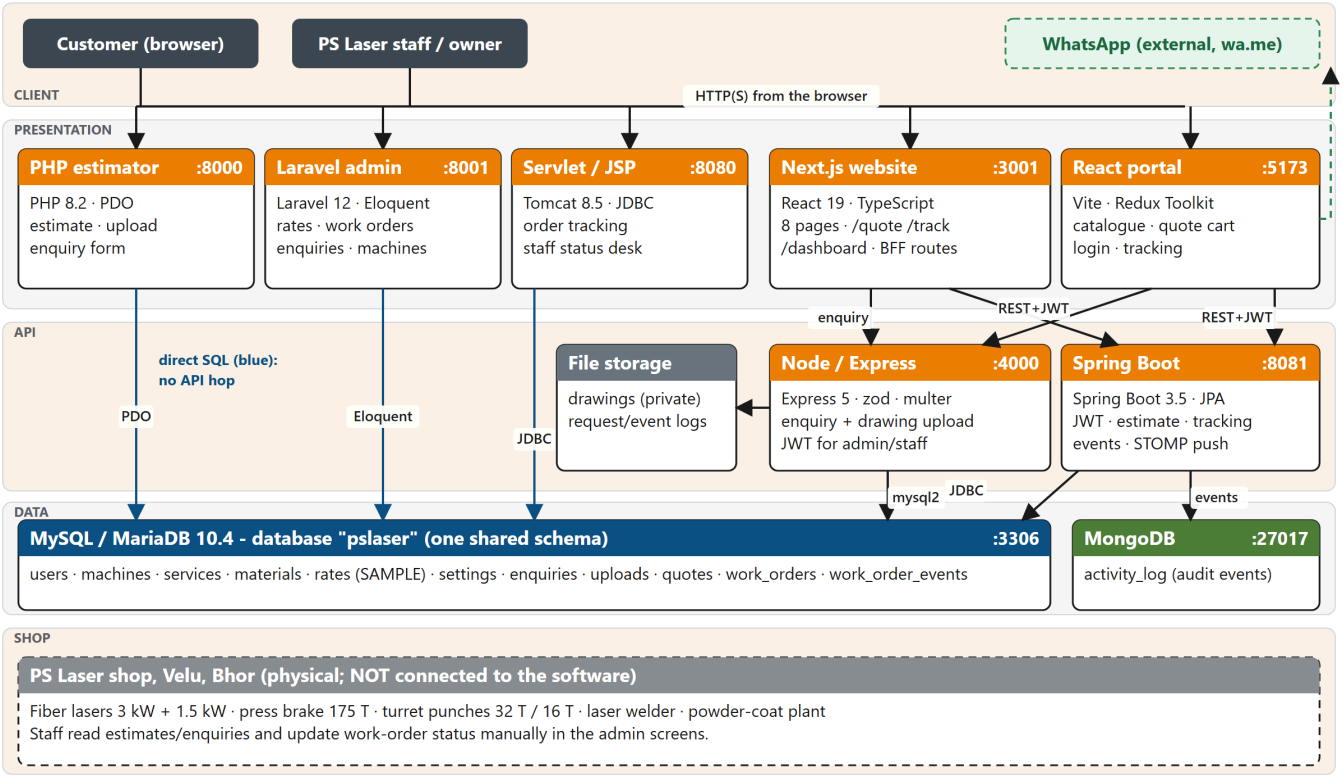



Fig. 1. PS Laser system architecture as built and run locally (ports as run in October 2026). Black arrows are HTTP/JSON calls; blue arrows are direct SQL access to the shared schema; the dashed green arrow is the WhatsApp deep link.

### C. Data model

One MySQL database, `pslaser`, holds eleven shared tables: `users`, `machines`, `services`, `materials`, `rates`, `settings`, `enquiries`, `uploads`, `quotes`, `work_orders` and `work_order_events`, plus framework-owned tables created by Laravel. Rates are stored per material and thickness band, and a `settings` row marks the rate card as sample data. A work order moves through seven stages: received, cutting, bending, welding, coating, inspection and dispatched. Every change appends a row to `work_order_events`, which is what the customer sees as a timeline.

### D. Cost model

Inputs are the material code, thickness $t$ (mm), part length $L$ and width $W$ (mm), cut length $\ell$ (mm), pierces $n_p$, bends $n_b$, a powder-coat flag and quantity $q$. After validation (the material exists, $0 < t \le$ its maximum, quantity $\ge 1$, and the part fits the 3000 × 1500 mm bed), the rate row whose thickness band contains $t$ is selected. The estimate is:

$$A = L \cdot W / 10^6 \text{ [m²]}, \; m = A \cdot t \cdot \rho \text{ [kg]} \quad (1)$$

$$C_{mat} = m \cdot r_{kg}, \; C_{cut} = (\ell/1000) \cdot r_{cut}, \; C_{prc} = n_p \cdot r_p, \; C_{bnd} = n_b \cdot r_b \quad (2)$$

$$C_{coat} = 2 \cdot A \cdot r_c \text{ (powder coat only, otherwise 0)} \quad (3)$$

$$P = \Sigma \, \text{round}_2(C_i), \; S = \text{round}_2(P \cdot q), \; G = \text{round}_2(S \cdot g/100), \; T = S + G \quad (4)$$

Here ρ is the material density in g/cm³ (so $A \cdot t \cdot \rho$ is in kg), $r_{kg}$, $r_{cut}$ and $r_p$ come from the selected rate row, $r_b$ and $r_c$ are the bend and powder-coat rates, $g$ is the GST percentage (18 % in the seed data), and $\text{round}_2$ rounds half-up to two decimals. The rule "round each line, then add" is the single rule that keeps the four implementations identical to the paisa. The seed rate card is **sample data** (flag `is_sample = 1`), so every estimate is labelled "Indicative estimate; final quote on WhatsApp".

### E. Security design

- **Authentication.** Spring Boot issues an HS256 JWT (8 h) that Node.js also accepts because both services share one secret. Roles are admin, staff and customer. The Next.js site keeps the token in an httpOnly, SameSite=Lax cookie that browser JavaScript cannot read.
- **Passwords.** Stored as bcrypt (`$2y$`) hashes in the shared `users` table.
- **Input handling.** Node.js validates bodies with zod and uses prepared statements; PHP and Laravel use CSRF tokens (a missing token yields HTTP 303 or 419); the estimate API rejects out-of-range inputs with HTTP 400 and a field-level error body.
- **Uploads.** Allowed types are DXF, DWG, PDF, PNG and JPG up to 10 MB; files are stored privately and served only to authenticated staff.
- **Abuse control.** The enquiry endpoint is rate-limited (HTTP 429), Node.js uses the helmet header set, and all APIs accept only an allow-list of origins.

- **Privacy in tracking.** An order is shown only if the order number and phone number both match, and the phone number is displayed masked.

### *F. Live updates and offline resilience*

When staff change a job status, Spring Boot saves the change and an event row, then publishes a message on the STOMP topic `/topic/work-orders` that the Next.js staff dashboard subscribes to. The public Next.js estimator calls Spring through its own route handler; if Spring is unreachable, the same formula is computed in TypeScript and the page shows "Offline estimate (sample)". The React portal similarly falls back to demo data and says so.

## V. IMPLEMENTATION

Table I lists the implemented services, which follow the vendors' documentation [10]–[12]. Version numbers are those declared in the project files or recorded on the development machine (Windows 11, Intel Core i5-12450H, 15.6 GB RAM).

TABLE I. IMPLEMENTED SERVICES AND TECHNOLOGY

| Service (port) | Technology | Role |
|---|---|---|
| Next.js site (3001) | Next.js 16.3, React 19, TypeScript, Tailwind 4, Zustand | Public site, /quote, /track, /dashboard |
| React portal (5173) | Vite, React 19, Redux Toolkit, React Router | Customer portal |
| PHP estimator (8000) | PHP 8.2, PDO, Bootstrap/jQuery | Estimate, upload, enquiry |
| Laravel admin (8001) | Laravel 12, Eloquent | Back office |
| Servlet/JSP (8080) | Tomcat 8.5, Java 21, JSTL, JDBC | Tracking, status desk |
| Spring API (8081) | Spring Boot 3.5, Java 21, JPA, JWT, STOMP | Main REST API |
| Node API (4000) | Node 24, Express 5, zod, multer, helmet | Enquiry + upload |
| Data | MySQL/MariaDB 10.4; MongoDB (embedded) | Shared schema; audit log |

The Spring service is layered as controller → service → repository, uses constructor injection, and publishes domain events that a listener turns into MongoDB activity records. Its health endpoint includes a custom rate-card indicator. The Next.js site mixes server components, a server action that posts enquiries to Node.js, a route handler that proxies estimates to Spring, and client components for the calculator and dashboard; a debounce hook (350 ms) limits estimate requests while the user types. Table II maps each syllabus unit to the feature that demonstrates it.

TABLE II. CS3111 UNITS AND THE PS LASER FEATURE THAT COVERS THEM

| Unit | Technology | Feature in PS Laser |
|---|---|---|
| I | PHP, Laravel | Estimator, drawing upload, enquiry; admin back office |
| II | React, Router, Flux, Redux | Customer portal: catalogue, quote cart, login |
| III | Servlet, JSP | Order tracking and staff status desk |
| IV | Spring Boot | REST API: JWT, JPA, MongoDB, events |
| V | Advanced React | TypeScript site: /quote, /track, /dashboard |
| VI | Node.js, Express | Enquiry + upload API, logging, static docs |

## VI. RESULTS AND VERIFICATION

We report only checks that were actually executed. Dates are those on which a check was run; the longer QA run was on 14 October 2026 and the re-runs below on 16 and 18 October 2026.

### *A. Reference estimate*

The reference part is mild steel, 2 mm, 500 × 300 mm, cut length 1600 mm, 4 pierces, 2 bends, powder-coated, quantity 10. With the sample rates stored in the database (75.00 INR/kg, 10.00 INR/m, 1.00 INR per pierce, 8 INR per bend, 120 INR/m² coating, 18 % GST) the Spring API returned the breakdown in Table III, which agrees with a hand calculation: $A$ = 0.15 m², $m$ = 2.355 kg, material 176.625 → 176.63.

TABLE III. REFERENCE ESTIMATE RETURNED BY THE SPRING API (16 OCT 2026, SAMPLE RATES)

| Component | INR | Component | INR |
|---|---|---|---|
| Material | 176.63 | Per part | 248.63 |
| Cutting | 16.00 | Subtotal (×10) | 2,486.30 |
| Piercing | 4.00 | GST 18 % | 447.53 |
| Bending | 16.00 | **Total** | **2,933.83** |
| Powder coat | 36.00 | isSample | true |

### *B. Cross-stack equivalence and functional checks*

On 14 October 2026 the same reference input gave the same total of 2,933.83 INR on the Spring API, the Next.js route handler, the PHP estimator and the Java servlet application. Further checks are listed in Table IV.

TABLE IV. EXECUTED CHECKS AND OUTCOMES

| Check | Outcome | Date |
|---|---|---|
| All services answer HTTP 200 (3001, 4000, 5173, 8000, 8001, 8080, 8081) | Pass | 14, 16 Oct |
| 14 site URLs return 200; unknown URL returns 404 | Pass | 14 Oct |
| Thickness above material limit (MS 20 mm) | HTTP 400 | 14 Oct |
| Tracking with correct / wrong phone number | 200 / 404 | 14 Oct |
| Admin endpoints without token (Spring, Node) | HTTP 401 | 14 Oct |
| Token issued by Spring accepted by Node | Pass (200) | 14 Oct |
| PHP bad CSRF token / Laravel POST without CSRF | 303 / 419 | 14 Oct |
| Laravel login and 7 admin screens | Pass | 14 Oct |
| Spring health with embedded MongoDB | UP | 16 Oct |
| Node.js automated suite (33 tests, live MySQL) | 33 passed | 18 Oct |
| Laravel automated suite (in-memory SQLite) | 8 passed (47 assertions) | 18 Oct |
| Production build, lint and type check of the Next.js app | Pass | 14 Oct |

The Spring project contains 15 automated test methods (unit tests with mocked repositories, a web-layer test and an integration test on an in-memory H2 database). They were counted from source and reported as passing by the build agents, but we did **not** re-run them for this paper.

### C. Not verified

The following were not tested, and we make no claim about them: responsive layout at 375, 768 and 1440 px by visual inspection; every image on every page; the end-to-end path from the React portal through Node.js to the database; delivery of a live STOMP push to the dashboard; re-testing of upload rejection with real files; any load or performance measurement; and any deployment outside a single Windows PC. In particular, no latency or throughput figures are reported because none were measured.

## VII. DISCUSSION AND LIMITATIONS

- **Sample prices.** The rate card is placeholder data. Estimates must not be used to quote customers until the owner enters real rates and clears the sample flag.
- **Duplicated rules.** Because three applications use the database directly, a rule change (for example in rounding) must be made in four places. The shared contract and the reference test case mitigate this but do not remove it.
- **No machine integration.** Job status is entered by staff; there is no connection to the lasers, press brake or punches, and no CAD/DXF analysis. Cut length and pierce count are typed by the user.
- **Delivery channels.** Enquiries produce a database row and a WhatsApp link; there is no e-mail or SMS service and no payment.
- **Deployment and secrets.** The system runs locally with development defaults (database root user with empty password, development JWT secret). These must be replaced before any public deployment, which would also need HTTPS and backups.
- **Data durability.** The embedded MongoDB instance loses data on restart.
- **Evidence base.** Results come from one development machine and a small seeded data set; they demonstrate correctness and consistency, not scalability.

## VIII. CONCLUSION AND FUTURE WORK

We designed and implemented PS Laser, a platform that gives a laser-cutting job shop an instant indicative estimator, enquiry and drawing capture, order tracking and staff tools, built as seven cooperating services over one shared schema and contract. The reference estimate matched across four independent implementations, and the automated suites that we re-ran (Node.js and Laravel) passed. Future work is to replace the sample rate card with the owner's real rates, add e-mail or WhatsApp Business notifications on status change, deploy behind HTTPS with managed MySQL and secrets, add end-to-end browser tests, parse DXF files to propose cut length and pierces automatically, and evaluate the estimator against the shop's actual quotations.


## ACKNOWLEDGMENT

The authors thank PS Laser (Velu, Bhor) for the company profile on which the machine and service information is based, and the faculty of the CS3111 course for guidance.


## REFERENCES


1. R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.
2. G. E. Krasner and S. T. Pope, "A cookbook for using the model-view-controller user interface paradigm in Smalltalk-80," *J. Object-Oriented Program.*, vol. 1, no. 3, pp. 26–49, Aug./Sep. 1988.
3. M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Internet Eng. Task Force, RFC 7519, May 2015.
4. M. Jones, "JSON Web Algorithms (JWA)," Internet Eng. Task Force, RFC 7518, May 2015.
5. N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf., FREENIX Track*, Monterey, CA, USA, 1999, pp. 81–91.
6. I. Fette and A. Melnikov, "The WebSocket protocol," Internet Eng. Task Force, RFC 6455, Dec. 2011.
7. OWASP Foundation, "OWASP Top 10:2021." [Online]. Available: https://owasp.org/Top10/
8. *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*, ISO/IEC 25010:2011, 2011.
9. W. M. Steen and J. Mazumder, *Laser Material Processing*, 4th ed. London, U.K.: Springer, 2010.
10. *Spring Boot Reference Documentation*, version 3.5. [Online]. Available: https://docs.spring.io/spring-boot/
11. *Next.js Documentation*, Vercel Inc. [Online]. Available: https://nextjs.org/docs
12. *Laravel Documentation*, version 12.x. [Online]. Available: https://laravel.com/docs/12.x